

Image classification

 Run in Google Colab (<https://colab.research.google.com/github/tensorflow/docs/blob/master/site/en/>)

 View source on GitHub (<https://github.com/tensorflow/docs/blob/master/site/en/tutorials/images/cla>)

 Download notebook (https://storage.googleapis.com/tensorflow_docs/docs/site/en/tutorials/images/c)

This tutorial shows how to classify images of flowers using a `tf.keras.Sequential` (https://www.tensorflow.org/api_docs/python/tf/keras/Sequential) model and load data using `tf.keras.utils.image_dataset_from_directory` (https://www.tensorflow.org/api_docs/python/tf/keras/preprocessing/image_dataset_from_directory). It demonstrates the following concepts:

- Efficiently loading a dataset off disk.
- Identifying overfitting and applying techniques to mitigate it, including data augmentation and dropout.

This tutorial follows a basic machine learning workflow:

1. Examine and understand data
2. Build an input pipeline
3. Build the model
4. Train the model
5. Test the model
6. Improve the model and repeat the process

In addition, the notebook demonstrates how to convert a [saved model](https://www.tensorflow.org/guide/saved_model) (https://www.tensorflow.org/guide/saved_model) to a [TensorFlow Lite](https://www.tensorflow.org/lite/) (<https://www.tensorflow.org/lite/>) model for on-device machine learning on mobile, embedded, and IoT devices.

Setup

Import TensorFlow and other necessary libraries:

```
import matplotlib.pyplot as plt
import numpy as np
import PIL
import tensorflow as tf

from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.models import Sequential
```

Download and explore the dataset

This tutorial uses a dataset of about 3,700 photos of flowers. The dataset contains five sub-directories, one per class:

```
flower_photo/
  daisy/
  dandelion/
  roses/
  sunflowers/
  tulips/
```

```
import pathlib
```

```
dataset_url = "https://storage.googleapis.com/download.tensorflow.org/example_images/flower_photos.tar"
data_dir = tf.keras.utils.get_file('flower_photos.tar', origin=dataset_url, extract=True)
data_dir = pathlib.Path(data_dir).with_suffix('')
```

After downloading, you should now have a copy of the dataset available. There are 3,670 total images:

```
image_count = len(list(data_dir.glob('*/*.jpg')))  
print(image_count)
```

Here are some roses:

```
roses = list(data_dir.glob('roses/*'))  
PIL.Image.open(str(roses[0]))
```

```
PIL.Image.open(str(roses[1]))
```

And some tulips:

```
tulips = list(data_dir.glob('tulips/*'))  
PIL.Image.open(str(tulips[0]))
```

```
PIL.Image.open(str(tulips[1]))
```

Load data using a Keras utility

Next, load these images off disk using the helpful

`tf.keras.utils.image_dataset_from_directory`

(https://www.tensorflow.org/api_docs/python/tf/keras/preprocessing/image_dataset_from_directory)

utility. This will take you from a directory of images on disk to a **`tf.data.Dataset`**

(https://www.tensorflow.org/api_docs/python/tf/data/Dataset) in just a couple lines of code. If you

like, you can also write your own data loading code from scratch by visiting the **Load and**

preprocess images (https://www.tensorflow.org/tutorials/load_data/images) tutorial.

Create a dataset

Define some parameters for the loader:

```
batch_size = 32
img_height = 180
img_width = 180
```

It's good practice to use a validation split when developing your model. Use 80% of the images for training and 20% for validation.

```
train_ds = tf.keras.utils.image_dataset_from_directory(
    data_dir,
    validation_split=0.2,
    subset="training",
    seed=123,
    image_size=(img_height, img_width),
    batch_size=batch_size)
```

```
val_ds = tf.keras.utils.image_dataset_from_directory(
    data_dir,
    validation_split=0.2,
    subset="validation",
    seed=123,
    image_size=(img_height, img_width),
    batch_size=batch_size)
```

You can find the class names in the `class_names` attribute on these datasets. These correspond to the directory names in alphabetical order.

```
class_names = train_ds.class_names
print(class_names)
```

Visualize the data

Here are the first nine images from the training dataset:

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 10))
for images, labels in train_ds.take(1):
    for i in range(9):
        ax = plt.subplot(3, 3, i + 1)
        plt.imshow(images[i].numpy().astype("uint8"))
        plt.title(class_names[labels[i]])
        plt.axis("off")
```

You will pass these datasets to the Keras [Model.fit](#)

(https://www.tensorflow.org/api_docs/python/tf/keras/Model#fit) method for training later in this tutorial. If you like, you can also manually iterate over the dataset and retrieve batches of images:

```
for image_batch, labels_batch in train_ds:
    print(image_batch.shape)
    print(labels_batch.shape)
    break
```

The `image_batch` is a tensor of the shape (32, 180, 180, 3). This is a batch of 32 images of shape 180x180x3 (the last dimension refers to color channels RGB). The `label_batch` is a tensor of the shape (32,), these are corresponding labels to the 32 images.

You can call `.numpy()` on the `image_batch` and `labels_batch` tensors to convert them to a `numpy.ndarray`.

Configure the dataset for performance

Make sure to use buffered prefetching, so you can yield data from disk without having I/O become blocking. These are two important methods you should use when loading data:

- **Dataset.cache** (https://www.tensorflow.org/api_docs/python/tf/data/Dataset#cache) keeps the images in memory after they're loaded off disk during the first epoch. This will ensure the dataset does not become a bottleneck while training your model. If your dataset is too large to fit into memory, you can also use this method to create a performant on-disk cache.
- **Dataset.prefetch** (https://www.tensorflow.org/api_docs/python/tf/data/Dataset#prefetch) overlaps data preprocessing and model execution while training.

Interested readers can learn more about both methods, as well as how to cache data to disk in the *Prefetching* section of the Better performance with the tf.data API (https://www.tensorflow.org/guide/data_performance) guide.

```
AUTOTUNE = tf.data.AUTOTUNE
```

```
train_ds = train_ds.cache().shuffle(1000).prefetch(buffer_size=AUTOTUNE)
val_ds = val_ds.cache().prefetch(buffer_size=AUTOTUNE)
```

Standardize the data

The RGB channel values are in the `[0, 255]` range. This is not ideal for a neural network; in general you should seek to make your input values small.

Here, you will standardize values to be in the `[0, 1]` range by using

tf.keras.layers.Rescaling (https://www.tensorflow.org/api_docs/python/tf/keras/layers/Rescaling) :

```
normalization_layer = layers.Rescaling(1./255)
```

There are two ways to use this layer. You can apply it to the dataset by calling **Dataset.map** (https://www.tensorflow.org/api_docs/python/tf/data/Dataset#map):

```
normalized_ds = train_ds.map(lambda x, y: (normalization_layer(x), y))
image_batch, labels_batch = next(iter(normalized_ds))
```

```
first_image = image_batch[0]
# Notice the pixel values are now in `[0,1]`.
print(np.min(first_image), np.max(first_image))
```

Or, you can include the layer inside your model definition, which can simplify deployment. Use the second approach here.

Note: You previously resized images using the `image_size` argument of

`tf.keras.utils.image_dataset_from_directory`

(https://www.tensorflow.org/api_docs/python/tf/keras/preprocessing/image_dataset_from_directory). If you want to include the resizing logic in your model as well, you can use the `tf.keras.layers.Resizing` (https://www.tensorflow.org/api_docs/python/tf/keras/layers/Resizing) layer.

A basic Keras model

Create the model

The Keras `Sequential` (https://www.tensorflow.org/guide/keras/sequential_model) model consists of three convolution blocks (`tf.keras.layers.Conv2D`

(https://www.tensorflow.org/api_docs/python/tf/keras/layers/Conv2D)) with a max pooling layer

(`tf.keras.layers.MaxPooling2D`

(https://www.tensorflow.org/api_docs/python/tf/keras/layers/MaxPool2D)) in each of them. There's a fully-connected layer (`tf.keras.layers.Dense`

(https://www.tensorflow.org/api_docs/python/tf/keras/layers/Dense)) with 128 units on top of it that is activated by a ReLU activation function ('relu'). This model has not been tuned for high accuracy; the goal of this tutorial is to show a standard approach.

```
num_classes = len(class_names)
```

```
model = Sequential([
    layers.Rescaling(1./255, input_shape=(img_height, img_width, 3)),
    layers.Conv2D(16, 3, padding='same', activation='relu'),
    layers.MaxPooling2D(),
    layers.Conv2D(32, 3, padding='same', activation='relu'),
    layers.MaxPooling2D(),
    layers.Conv2D(64, 3, padding='same', activation='relu'),
```

```
layers.MaxPooling2D(),  
layers.Flatten(),  
layers.Dense(128, activation='relu'),  
layers.Dense(num_classes)  
])
```

Compile the model

For this tutorial, choose the [tf.keras.optimizers.Adam](https://www.tensorflow.org/api_docs/python/tf/keras/optimizers/Adam)

(https://www.tensorflow.org/api_docs/python/tf/keras/optimizers/Adam) optimizer and

[tf.keras.losses.SparseCategoricalCrossentropy](https://www.tensorflow.org/api_docs/python/tf/keras/losses/SparseCategoricalCrossentropy)

(https://www.tensorflow.org/api_docs/python/tf/keras/losses/SparseCategoricalCrossentropy) loss

function. To view training and validation accuracy for each training epoch, pass the `metrics` argument to [Model.compile](https://www.tensorflow.org/api_docs/python/tf/keras/Model#compile) (https://www.tensorflow.org/api_docs/python/tf/keras/Model#compile).

```
model.compile(optimizer='adam',  
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True,  
                        metrics=['accuracy']))
```

Model summary

View all the layers of the network using the Keras [Model.summary](https://www.tensorflow.org/api_docs/python/tf/keras/Model#summary)

(https://www.tensorflow.org/api_docs/python/tf/keras/Model#summary) method:

```
model.summary()
```

Train the model

Train the model for 10 epochs with the Keras [Model.fit](https://www.tensorflow.org/api_docs/python/tf/keras/Model#fit)

(https://www.tensorflow.org/api_docs/python/tf/keras/Model#fit) method:

```
epochs=10  
history = model.fit(  
    # Data  
    # Validation data  
    # Callbacks  
    # Early stopping  
    # TensorBoard
```



```
train_ds,  
validation_data=val_ds,  
epochs=epochs  
)
```

Visualize training results

Create plots of the loss and accuracy on the training and validation sets:

```
acc = history.history['accuracy']  
val_acc = history.history['val_accuracy']  
  
loss = history.history['loss']  
val_loss = history.history['val_loss']  
  
epochs_range = range(epochs)  
  
plt.figure(figsize=(8, 8))  
plt.subplot(1, 2, 1)  
plt.plot(epochs_range, acc, label='Training Accuracy')  
plt.plot(epochs_range, val_acc, label='Validation Accuracy')  
plt.legend(loc='lower right')  
plt.title('Training and Validation Accuracy')  
  
plt.subplot(1, 2, 2)  
plt.plot(epochs_range, loss, label='Training Loss')  
plt.plot(epochs_range, val_loss, label='Validation Loss')  
plt.legend(loc='upper right')  
plt.title('Training and Validation Loss')  
plt.show()
```

The plots show that training accuracy and validation accuracy are off by large margins, and the model has achieved only around 60% accuracy on the validation set.

The following tutorial sections show how to inspect what went wrong and try to increase the overall performance of the model.

Overfitting

In the plots above, the training accuracy is increasing linearly over time, whereas validation accuracy stalls around 60% in the training process. Also, the difference in accuracy between training and validation accuracy is noticeable—a sign of overfitting

(https://www.tensorflow.org/tutorials/keras/overfit_and_underfit).

When there are a small number of training examples, the model sometimes learns from noises or unwanted details from training examples—to an extent that it negatively impacts the performance of the model on new examples. This phenomenon is known as overfitting. It means that the model will have a difficult time generalizing on a new dataset.

There are multiple ways to fight overfitting in the training process. In this tutorial, you'll use *data augmentation* and add *dropout* to your model.

Data augmentation

Overfitting generally occurs when there are a small number of training examples. Data augmentation (https://www.tensorflow.org/tutorials/images/data_augmentation) takes the approach of generating additional training data from your existing examples by augmenting them using random transformations that yield believable-looking images. This helps expose the model to more aspects of the data and generalize better.

You will implement data augmentation using the following Keras preprocessing layers:

`tf.keras.layers.RandomFlip`

(https://www.tensorflow.org/api_docs/python/tf/keras/layers/RandomFlip),

`tf.keras.layers.RandomRotation`

(https://www.tensorflow.org/api_docs/python/tf/keras/layers/RandomRotation), and

`tf.keras.layers.RandomZoom`

(https://www.tensorflow.org/api_docs/python/tf/keras/layers/RandomZoom). These can be included inside your model like other layers, and run on the GPU.

```
data_augmentation = keras.Sequential(  
    [  
        layers.RandomFlip("horizontal",  
                           input_shape=(img_height,  
                                         img_width,
```

```

        3)),
        layers.RandomRotation(0.1),
        layers.RandomZoom(0.1),
    ]
)

```

Visualize a few augmented examples by applying data augmentation to the same image several times:

```

plt.figure(figsize=(10, 10))
for images, _ in train_ds.take(1):
    for i in range(9):
        augmented_images = data_augmentation(images)
        ax = plt.subplot(3, 3, i + 1)
        plt.imshow(augmented_images[0].numpy().astype("uint8"))
        plt.axis("off")

```

You will add data augmentation to your model before training in the next step.

Dropout

Another technique to reduce overfitting is to introduce dropout (https://developers.google.com/machine-learning/glossary#dropout_regularization) regularization to the network.

When you apply dropout to a layer, it randomly drops out (by setting the activation to zero) a number of output units from the layer during the training process. Dropout takes a fractional number as its input value, in the form such as 0.1, 0.2, 0.4, etc. This means dropping out 10%, 20% or 40% of the output units randomly from the applied layer.

Create a new neural network with `tf.keras.layers.Dropout` (https://www.tensorflow.org/api_docs/python/tf/keras/layers/Dropout) before training it using the augmented images:

```

model = Sequential([
    data_augmentation,

```

```
layers.Rescaling(1./255),
layers.Conv2D(16, 3, padding='same', activation='relu'),
layers.MaxPooling2D(),
layers.Conv2D(32, 3, padding='same', activation='relu'),
layers.MaxPooling2D(),
layers.Conv2D(64, 3, padding='same', activation='relu'),
layers.MaxPooling2D(),
layers.Dropout(0.2),
layers.Flatten(),
layers.Dense(128, activation='relu'),
layers.Dense(num_classes, name="outputs")
])
```

Compile and train the model

```
model.compile(optimizer='adam',
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=['accuracy'])
```

```
model.summary()
```

```
epochs = 15
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=epochs
)
```

Visualize training results

After applying data augmentation and `tf.keras.layers.Dropout`

(https://www.tensorflow.org/api_docs/python/tf/keras/layers/Dropout), there is less overfitting than before, and training and validation accuracy are closer aligned:

```
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']

loss = history.history['loss']
val_loss = history.history['val_loss']

epochs_range = range(epochs)

plt.figure(figsize=(8, 8))
plt.subplot(1, 2, 1)
plt.plot(epochs_range, acc, label='Training Accuracy')
plt.plot(epochs_range, val_acc, label='Validation Accuracy')
plt.legend(loc='lower right')
plt.title('Training and Validation Accuracy')

plt.subplot(1, 2, 2)
plt.plot(epochs_range, loss, label='Training Loss')
plt.plot(epochs_range, val_loss, label='Validation Loss')
plt.legend(loc='upper right')
plt.title('Training and Validation Loss')
plt.show()
```

Predict on new data

Use your model to classify an image that wasn't included in the training or validation sets.

Note: Data augmentation and dropout layers are inactive at inference time.

```
sunflower_url = "https://storage.googleapis.com/download.tensorflow.org/example_
sunflower_path = tf.keras.utils.get_file('Red_sunflower', origin=sunflower_url)

img = tf.keras.utils.load_img(
    sunflower_path, target_size=(img_height, img_width))
```

```

)
img_array = tf.keras.utils.img_to_array(img)
img_array = tf.expand_dims(img_array, 0) # Create a batch

predictions = model.predict(img_array)
score = tf.nn.softmax(predictions[0])

print(
    "This image most likely belongs to {} with a {:.2f} percent confidence."
    .format(class_names[np.argmax(score)], 100 * np.max(score))
)

```

Use TensorFlow Lite

TensorFlow Lite is a set of tools that enables on-device machine learning by helping developers run their models on mobile, embedded, and edge devices.

Convert the Keras Sequential model to a TensorFlow Lite model

To use the trained model with on-device applications, first [convert it](https://www.tensorflow.org/lite/models/convert) (https://www.tensorflow.org/lite/models/convert) to a smaller and more efficient model format called a [TensorFlow Lite](https://www.tensorflow.org/lite/) (https://www.tensorflow.org/lite/) model.

In this example, take the trained Keras Sequential model and use

[tf.lite.TFLiteConverter.from_keras_model](https://www.tensorflow.org/api_docs/python/tf/lite/TFLiteConverter#from_keras_model)

(https://www.tensorflow.org/api_docs/python/tf/lite/TFLiteConverter#from_keras_model) to generate a [TensorFlow Lite](https://www.tensorflow.org/lite/) (https://www.tensorflow.org/lite/) model:

```

# Convert the model.
converter = tf.lite.TFLiteConverter.from_keras_model(model)
tflite_model = converter.convert()

# Save the model.
with open('model.tflite', 'wb') as f:
    f.write(tflite_model)

```

The TensorFlow Lite model you saved in the previous step can contain several function signatures. The Keras model converter API uses the default signature automatically. Learn more about [TensorFlow Lite signatures](https://www.tensorflow.org/lite/guide/signatures) (<https://www.tensorflow.org/lite/guide/signatures>).

Run the TensorFlow Lite model

You can access the TensorFlow Lite saved model signatures in Python via the [`tf.lite.Interpreter`](https://www.tensorflow.org/api_docs/python/tf/lite/Interpreter) (https://www.tensorflow.org/api_docs/python/tf/lite/Interpreter) class.

Load the model with the `Interpreter`:

```
TF_MODEL_FILE_PATH = 'model.tflite' # The default path to the saved TensorFlow L
interpreter = tf.lite.Interpreter(model_path=TF_MODEL_FILE_PATH)
```

Print the signatures from the converted model to obtain the names of the inputs (and outputs):

```
interpreter.get_signature_list()
```

In this example, you have one default signature called `serving_default`. In addition, the name of the 'inputs' is `sequential_1_input`, while the 'outputs' are called `outputs`. You can look up these first and last Keras layer names when running `Model.summary`, as demonstrated earlier in this tutorial.

Now you can test the loaded TensorFlow Model by performing inference on a sample image with [`tf.lite.Interpreter.get\_signature\_runner`](https://www.tensorflow.org/api_docs/python/tf/lite/Interpreter#get_signature_runner) (https://www.tensorflow.org/api_docs/python/tf/lite/Interpreter#get_signature_runner) by passing the signature name as follows:

```
classify_lite = interpreter.get_signature_runner('serving_default')
classify_lite
```

Similar to what you did earlier in the tutorial, you can use the TensorFlow Lite model to classify images that weren't included in the training or validation sets.

You have already tensorized that image and saved it as `img_array`. Now, pass it to the first argument (the name of the 'inputs') of the loaded TensorFlow Lite model (`predictions_lite`), compute softmax activations, and then print the prediction for the class with the highest computed probability.

```
predictions_lite = classify_lite(sequential_1_input=img_array)['outputs']
score_lite = tf.nn.softmax(predictions_lite)

print(
    "This image most likely belongs to {} with a {:.2f} percent confidence."
    .format(class_names[np.argmax(score_lite)], 100 * np.max(score_lite))
)
```

The prediction generated by the lite model should be almost identical to the predictions generated by the original model:

```
print(np.max(np.abs(predictions - predictions_lite)))
```

Of the five classes—'daisy', 'dandelion', 'roses', 'sunflowers', and 'tulips'—the model should predict the image belongs to sunflowers, which is the same result as before the TensorFlow Lite conversion.

Next steps

This tutorial showed how to train a model for image classification, test it, convert it to the TensorFlow Lite format for on-device applications (such as an image classification app), and perform inference with the TensorFlow Lite model with the Python API.

You can learn more about TensorFlow Lite through [tutorials](https://www.tensorflow.org/lite/tutorials) (<https://www.tensorflow.org/lite/tutorials>) and [guides](https://www.tensorflow.org/lite/guide) (<https://www.tensorflow.org/lite/guide>).

Except as otherwise noted, the content of this page is licensed under the [Creative Commons Attribution 4.0 License](https://creativecommons.org/licenses/by/4.0/) (<https://creativecommons.org/licenses/by/4.0/>), and code samples are licensed under the [Apache 2.0 License](https://www.apache.org/licenses/LICENSE-2.0) (<https://www.apache.org/licenses/LICENSE-2.0>). For details, see the [Google Developers Site Policies](https://developers.google.com/site-policies) (<https://developers.google.com/site-policies>). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2024-04-03 UTC.